

UNIT- 1

Chapter- 2

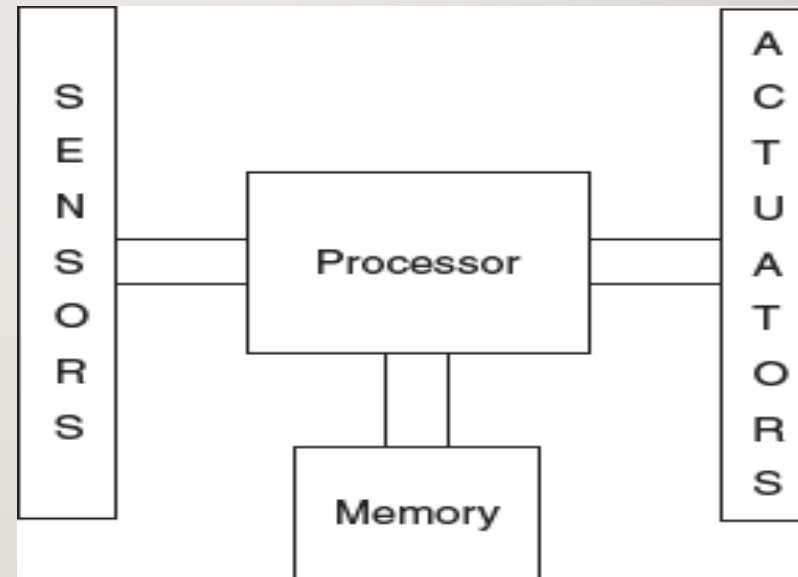
INTERNAL COMPONENTS OF A SYSTEM-ON-CHIP (SoC)

INTRODUCTION

- More focus will be towards the typical microcontroller units and its Internal units.
- Also look at the new trends that are taking place because having systems which are connected to the internet are becoming a necessity.

2.1 GENERAL MICROCONTROLLER UNIT

- Here, more focus is on hardware aspects of an embedded system.



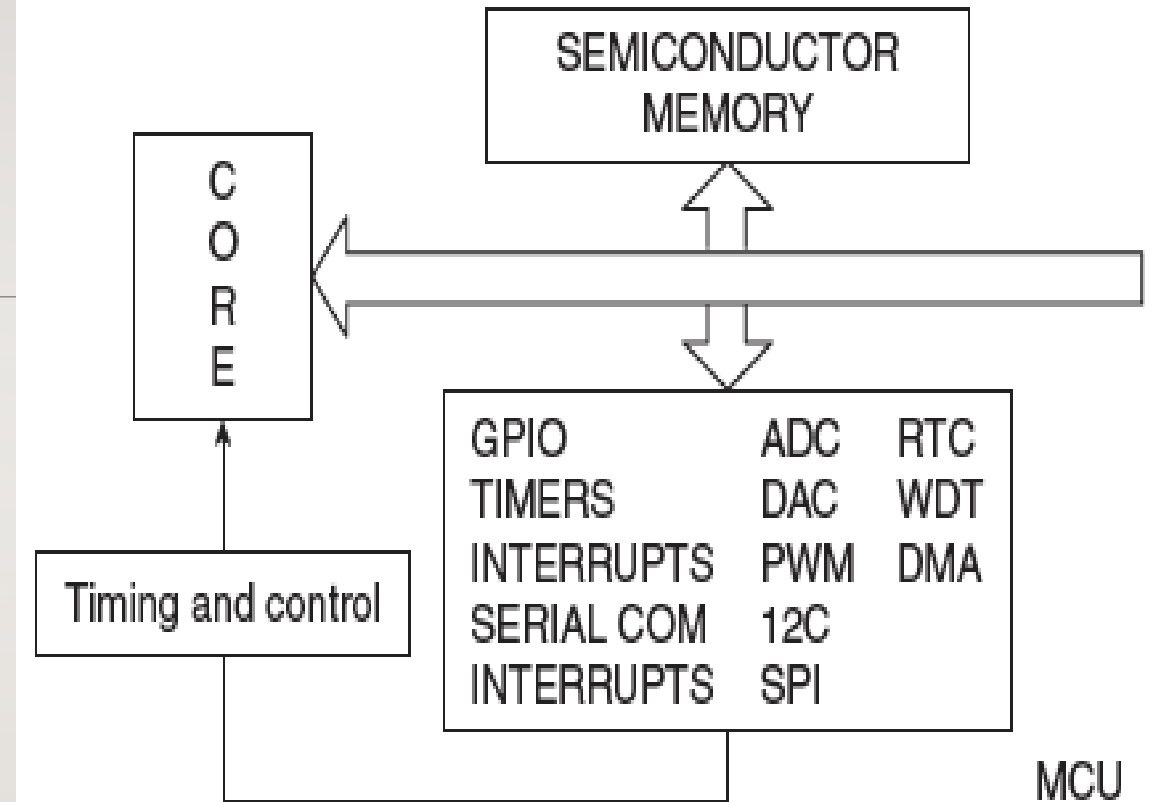
Model of a simple embedded system

- Embedded system has a computing unit to which sensors and actuators are connected.

- Here more focus on MCU which is a 'Single Chip Computer' and is a heart of embedded system.
- MCU is a computing engine, that is , a CPU, to which a peripheral controllers and memory are added.
- Peripherals: input and output devices which are needed for system.
- Eg: Keyboard as Input and LCD Screen as Output peripheral.
- As these are outside to MCU some are 'Controllers' that are inside the chip.
- Peripheral controllers are dedicated hardware units, that provides control signals for the MCU to communicate with actual peripheral devices.

Conceptual Internal diagram of an MCU chip.

- ❑ CORE: CPU or computing engine
- ❑ Memory and a set of peripherals interconnected by a system bus.
- ❑ These peripherals are found in most commonly in all MCU's.
- ❑ Advanced peripherals are found in more advanced system and all work on a basis of time synchronized manner, due to timers and control units.

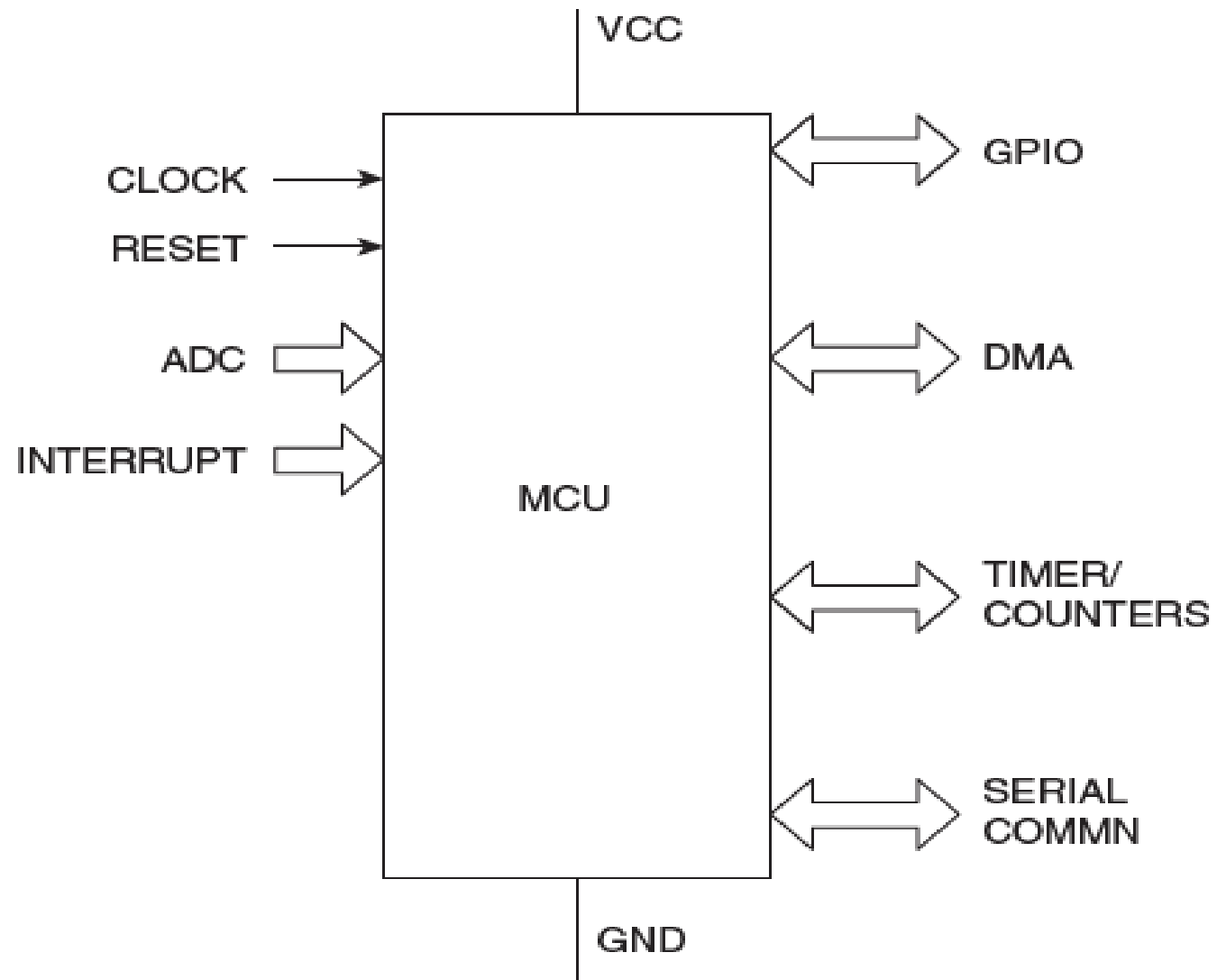


Internal Components of a Typical MCU

- Internal bus shown here is mainly consists of basic address, data and control bus.
- If the data bus has 16 bits, it means that 16 bits can be handled at a go by the CPU and the word length of the general purpose register in the core is also 16.
- The width of the internal address bus fixes an upper limit on the capacity of the internal memory that can be implemented.
- If it is of 16 bits, the maximum memory capacity is 64 KB
- If this bus is 32-Bits wide, the maximum addressable memory is 4 GB

2.2 PIN DIAGRAM

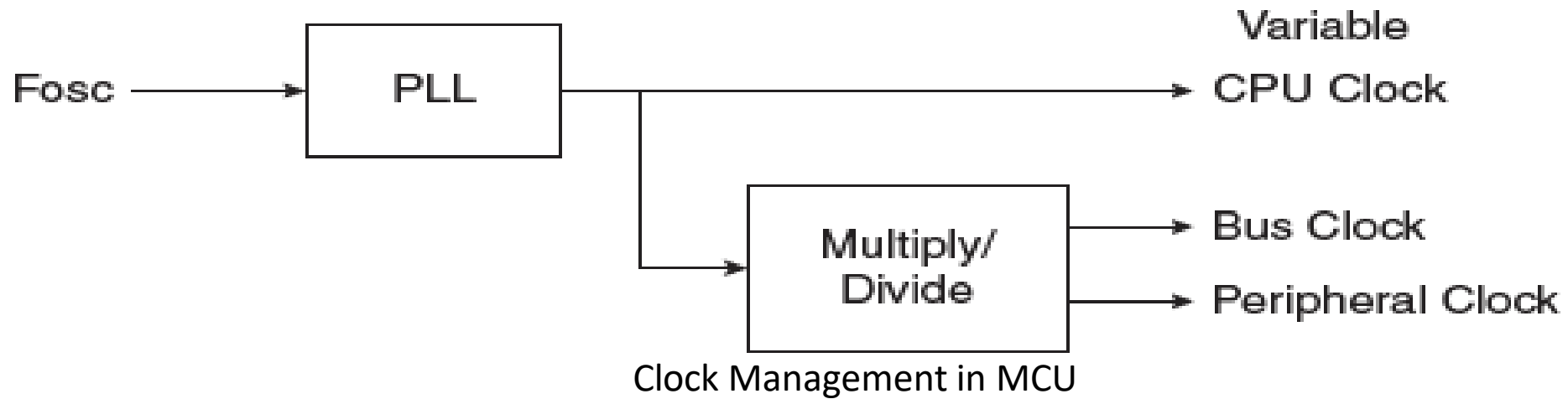
- Number of pins of an MCU depends on the number of peripherals it has.
- All peripherals has pins to send and receive signals to/from external devices.
- These pins are called I/O pins.
- In most of the MCU's , it has more than one function for a pin, such that the exact function can be configured as per the use.



Functional PIN Diagram of a MCU

2.2.1 CLOCK

- Every system is synchronized with a clock, designated as the system clock.
- The frequency of this clock is a measure of how fast the system can perform its functions with MCU, CORE, peripherals and the BUS which interconnect them.
- All of them can be synchronized by the same clock or each of them can have separate clock.
- Means it can have system clock, core clock, bus clock, peripheral clock- each of them operating at different frequencies.



- A crystal is used as basic reference for clock systems. Fosc(Frequency Oscillator)
- Most modern MCU, have the options of using either an external frequencies or the output of an internal oscillator as a basic clock.
- MCU has a circuitry, that supplies different frequencies for different part of the system.
- Frequencies above 100 MHz is needed then phase locked loop (PLL) is used as it can generate different frequencies in phase synchronization.
- All the stringent requirements needed for clocks such as rise/fall time, low propagation delay are realized using PLL.
- PLL can also change the CPU clock frequency dynamically for power management.

WHAT ARE THE ORDERS OF CLOCK FREQUENCIES GENERALLY USED IN EMBEDDED SYSTEMS?

- Embedded systems do not use very high frequencies of the order of 3GHz that we see in the latest PCs.
- Each embedded device has its specific functions and many applications require low frequencies.
- Core clock frequencies can be as low as 10MHz and
- Peripherals may operate at much lower rates.
- High-end microcontrollers have frequencies in the range of 60-80 MHz,
- Very high end side mobile phone and tablets which use frequencies of around 2 GHz.

2.2.2 RESET

- Word Reset means “Restart”.
- Systems are expected to start from the known state.
- As MCU is reset, internal registers are cleared and the first instruction to be executed is taken from a specific address.
- Hence, program counter should be loaded with that address.

HOW IS RESET DONE?

- Any MCU chip, there is a reset pin at which an external circuit may be connected.

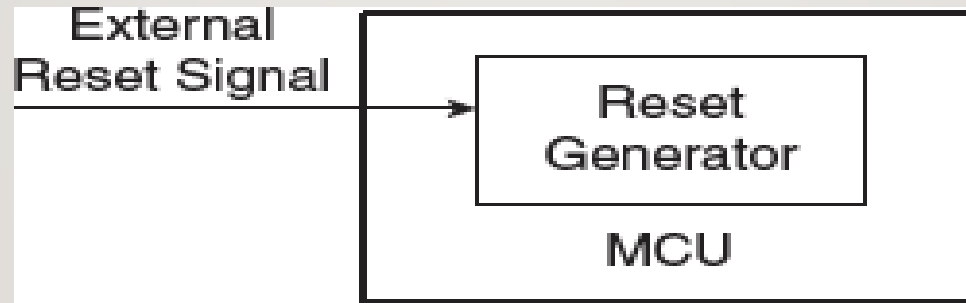


Fig: The Reset system

- Above figure, shows that there is an internal circuitry also, named as “Reset Generator”.
- That takes care of the reset function.
- There are 3 different kinds of reset options.

1. POWER-ON-RESET

- As power is first supplied to circuit, system must start from a known state.
- This is the state the MCU is in, every time it is powered on.
- The same state must also be reached if a hard reset is done by a push button switch any time during the operation of the circuit.
- Though an internal reset generator, signal from external circuit is mandatorily needed at the reset pin.
- This signal should appear at the reset pin whenever the power supply is switched on.
- This signal is termed a pulse that should remain at the reset pin.
- Resetting follow the circuit similar to Schmitt Trigger.

2. BROWN OUT RESET

- When the power supply goes below a certain pre-fixed value
- “Brown out” will occur
- Especially battery operated devices, its quite common high.
- When brownout occurs, the MCU starts behaving abnormally and gives wrong results.
- Leads to cause sudden destruction situations.
- To avoid this, all MCU's have the capability of reset, as it reaches brownout threshold.
- For any MCU's not having internal brownout capability, external circuit can be used to detect brownout and cause a reset to occur.

3. WATCH DOG RESET

- Most of the embedded systems are unsupervised and can go into infinite loops due to noise signals.
- To overcome this, in MCU there should be a ' Watch Dog' which initiates a 'reset' on break out of the infinite loop.
- This type of reset is also called a soft reset or a warm boot.

Feature	Power-On Reset (POR)	Brown-Out Reset (BOR)	Watchdog Reset (WDT)
Trigger Condition	When power is first applied	When supply voltage drops below safe level	When software fails to refresh watchdog timer
Cause	Power startup	Voltage dip (brown-out)	Program hang / infinite loop
Voltage Condition	0V → Normal Vcc	Normal Vcc → Below threshold	Voltage is normal
Purpose	Initialize system at startup	Protect during unstable/low voltage	Recover from software crash
Hardware/Software	Hardware-based	Hardware-based	Hardware timer monitored by software
When it Happens	Once at power-up	Any time voltage drops	Any time program stops responding
Used For	Proper system boot	Prevent data corruption	Improve system reliability



2.3.1 REGISTER OF THE CORE

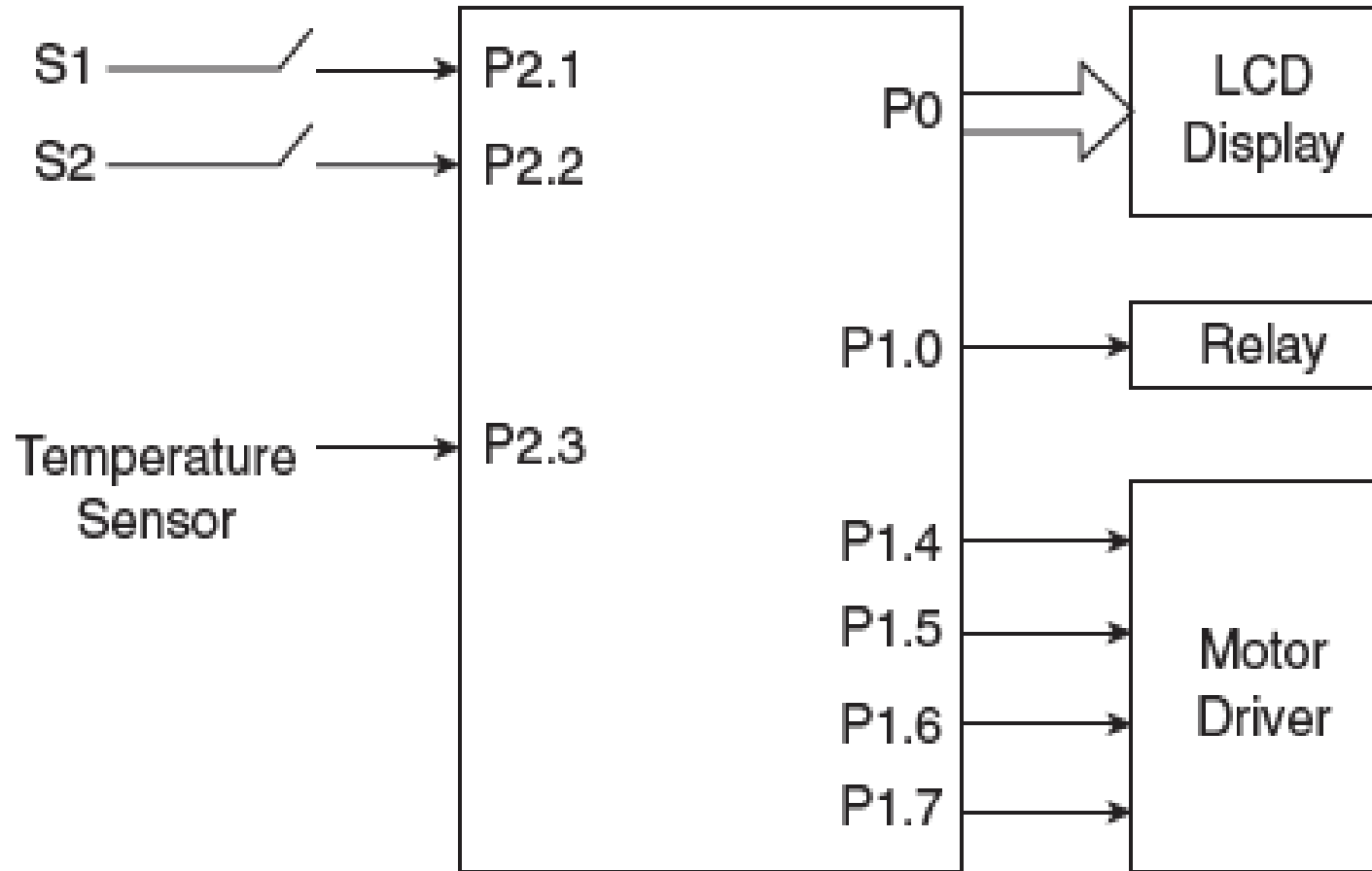
- Microcontroller has a CPU which we usually refer to core.
- Core has registers, that are used for computations.
- These registers are termed as “General Purpose Registers” OR “Scratchpad Registers”.
- These registers provide temporary storage for operands in computations.
- As core in an MCU is full fledged CPU, it has several registers, including general purpose registers, program counters, stack registers, flags etc.

2.3.2 REGISTERS OF PERIPHERALS

- For any MCU, peripherals are inside the chip. Denoted as 'On-Chip' peripherals.
- Means, while the processor is executing programs, many peripherals can work in parallel.
- For eg: take the case of a timer.
- The processor, through its instructions, only needs to tell a timer to start counting.
- Timer does not need any more intervention until it reaches the desired count.
- At that point, it will signal the processor by setting a flag or sending an 'Interrupt' and the processor then take any action.
- A number of peripherals run parallel, like ADC, DAC, Serial IO etc.

2.4.1 GENERAL PURPOSE I/O

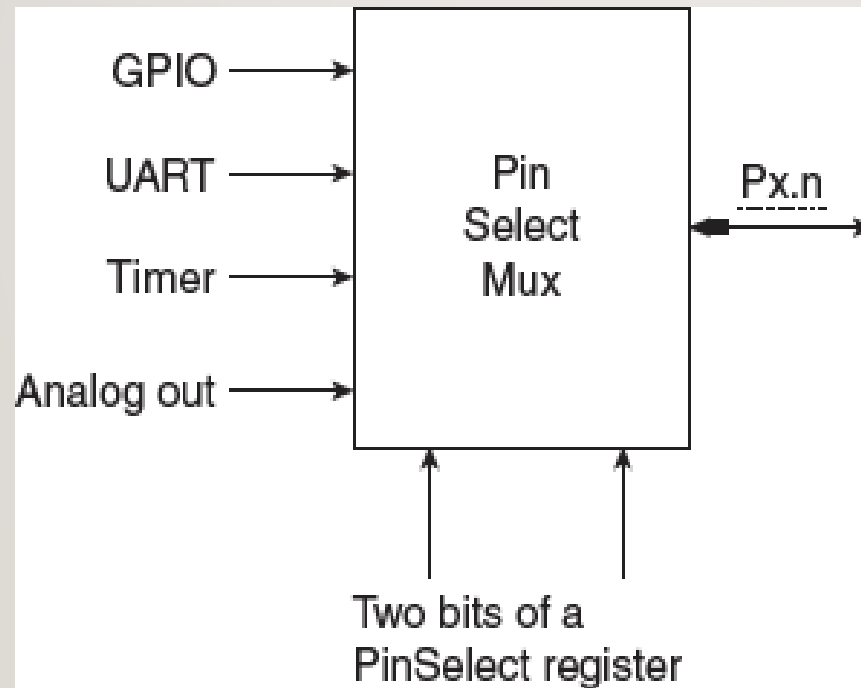
- MCU has a computing engine for processing
- As it needs to communicate with external world, it needs input/output pins.
- Input pins are used to take in for sensor data for processing
- Output pins for sending actuation signals to output devices.
- Those pins are called GPIO or “General Purpose Input Output pins”
- Pins can also be taken as groups or ports.
- Thus, an 8-bit MCU has 8-bit ports, with additional capability.



Using GPIO pins as Input & Output

-
- It shows a case of many GPIO pins used for various I/O functions.
 - An 8-bit port is configured as output and is utilized for sending 8-bit data to an LCD display.
 - one-bit pin is used for a relay
 - 4-bit output pins for sending commands to a motor driver IC.
 - Two-bit input pins are controlled by switches (S1 & S2)
 - Another one takes sensor data from a temperature sensor.
 - This illustrates that, it is possible to use GPIO pins as we need.

-
- In most MCUs, however, each pin has more than one function.



PIN selection function

-
- Lets see a typical pin, Pn.x
 - It is shown to have perform four functions, but only one of which may be chosen at a time.
 - This choice selection happens on the basis of multiplexing called multiplexer.
 - The select bit of the multiplexer are realized by two bits in a 'Pin Select' register.
 - First option of pin is using as GPIO, where it can be used either input or output pin
 - It works on the basis of set/reset.

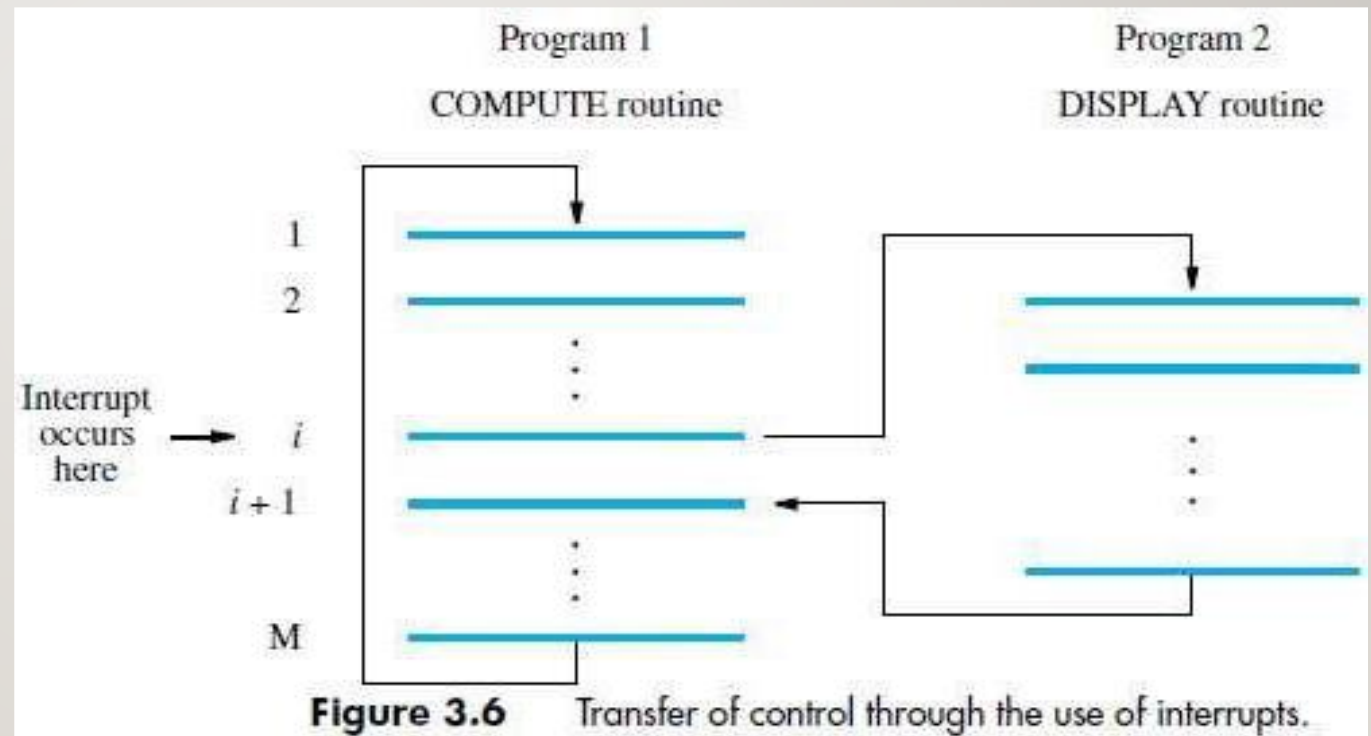
DATA TRANSFER MODES

- How does keyboard communicates with the MCU?
- To keep MCU 'in a loop' waiting for a key press.
- MCU has to continuously monitor this action on one of its input pins.
- This method of accepting an input data is called 'Polling'.
- MCU will be unable to do anything else if it is continuously polling some pin, in this fashion.
- Because of inefficiency involved, polling for data is rarely used in practical systems.
- So we use interrupts.

INTERRUPTS

- There are many situations where other tasks can be performed while waiting for an I/O device to become ready.
- The event that causes the interruption is called interrupt and the special routine executed to service the interrupt is called ISR.
- A hardware signal called an Interrupt will alert the processor when an I/O device becomes ready.
- Interrupt-signal is sent on the interrupt-request line.

- The processor can be performing its own task without the need to continuously check the I/O-device.
- The routine executed in response to an interrupt-request is called ISR.
- The processor must inform the device that its request has been recognized by sending INTA signal.
- For example, consider COMPUTE and
- PRINT routines (Figure 3.6).



-
- The processor first completes the execution of instruction i .
 - Then, the processor loads the PC with the address of the first instruction of the ISR.
 - After the execution of ISR, the processor has to come back to instruction $i+1$.
 - Therefore, when an interrupt occurs, the current content of the PC is put in the temporary storage location.
 - A return at the end of ISR reloads the PC from that temporary storage location.
 - This causes the execution to resume at instruction $i+1$.
 - When the processor is handling interrupts, it must inform the device that its request has been recognized.
 - This may be accomplished by INTA signal.

-
- The task of saving and restoring the information can be done automatically by the processor.
 - The processor saves only the contents of PC & Status register.
 - Saving registers also increases the Interrupt Latency.
 - **Interrupt Latency** is a delay between time an interrupt request is received and start the execution of the ISR
 - Generally, the long interrupt latency is unacceptable.

Subroutine	ISR
A subroutine performs a function required by the program from where it is called	ISR may not have anything in common with program being executed at time INTR is received.
Subroutine is just a linkage of 2 or more function related to each other.	Interrupt is a mechanism for coordinating I/O transfers.

INTERRUPTS

- 'Interrupts' is always get interrupted while we are busy working something important.
- Interrupt is a method, that allows MCU to do any activity until it gets a signal from the keyboard that is been pressed.
- This signal from the keyboard is called an 'Interrupt'.
- After receiving these signals, the MCU stops what it is doing and takes the action.
- If the processor decides to attend new task, it temporarily abandon the current task and takes up the new task, after completion, it resumes back.

-
- The processor executes a stream of instructions and the Program Counter (PC) is the register that sequences it.
 - At any point, PC points to the next instruction to be executed.
 - When an interrupt arrives, the process completes the current instruction and saves the context of the current task.
 - It save the address of the next instruction of this sequence

Is it possible for the processor to ignore interrupts?

- It is possible that sometimes, the processor does not care to be interrupted by certain peripherals.
- The interrupts from such peripherals may be masked selectively by using registers provided for the purpose.
- All processors have a Non Maskable Interrupt(NMI) which is used for tasks of great importance.

Why are interrupts very important in embedded systems?

- Many embedded systems are not active all the time.
- Mobile phones are inactive until a call comes or we want it to do something.
- So the best way to use such a device is to let it go to sleep and wake it up only when we need some service from it.
- That can be done very efficiently using the concept of interrupts.

TIMERS AND COUNTERS

- Timer/Counter is one of the part of peripherals of MCU.
- This is a dedicated hardware for timing applications.
- Reference clock for this hardware is the peripheral clock.
- Which is a low frequency clock derived from the core clock.
- It acts as a 'Interval Timer' or as an event counter.
- Interval counter creates a delay and by using this delay, a waveform can be generated at any output pin.

THE STEPS INVOLVED IN THE WORKING OF AN INTERVAL TIMER:

- 1) A timer count register is loaded with a number and the timer is started.
 - 2) The count increases/ decreases until it reaches the maximum/minimum count.
 - 3) At this point of time, the count register is cleared and a flag is set or an interrupt is triggered to signal the event of timer overflow.
- The time elapsed from the starting point, is the required delay.
 - This whole sequence can be repeated many times after re-loading the timer register.
 - If an output pin is toggled every time the timer register overflows, we get a symmetric square wave at the pin.

2.6.2 EVENT COUNTER

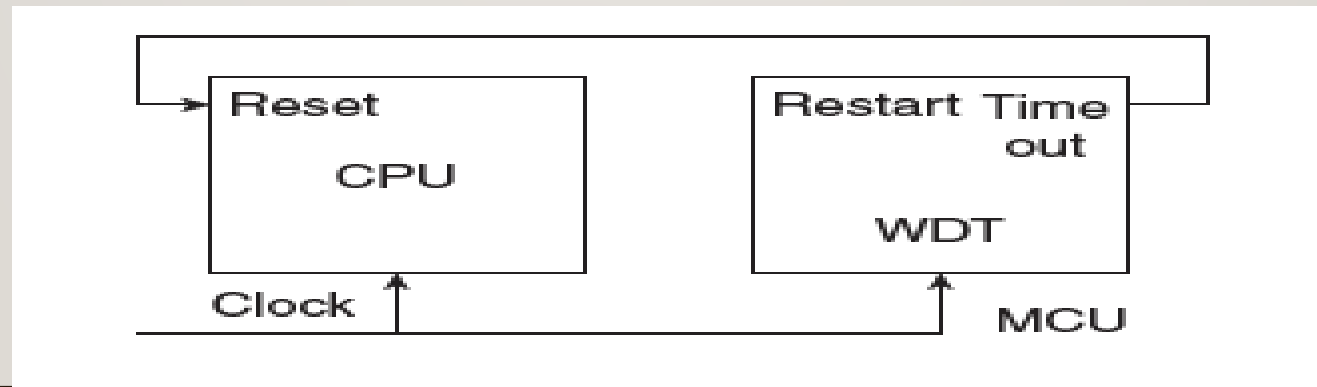
- The same hardware can be used to count external events.
- Lets say we need to measure the frequency of pulse train.
- This frequency can be fed into the counter input pin.
- Instead of peripheral clock being the reference, this unknown frequency acts as the reference clock.
- Every time a leading or trailing edge is sensed at the counter pin, the timer count register count increases by one.

WATCH DOG TIMERS

- This is an on-chip peripheral in advanced MCU's
- Low-end MCU's may not have it, can be added if needed, as chips are available.
- Embedded systems have to sort out their problems on their own, as they don't allow user intervention.
- A WDT is provided to help a system in the event of an unresolvable anomaly in its working.
- This timer monitors the working of the MCU to make sure it does not go into endless loops.
- When unexpected events occur, the only way out of it is to restart the system and start all over again.
- WDT is used to check whether such a RESET is necessary.

HOW DOES THE WDT DO THIS?

- It is like an ordinary timer
- Normally, it starts counting and before it reaches its terminal count, the internal mechanism restarts it.
- If the MCU stuck in the infinite loop, the 'Internal Mechanism' of restarting the WDT is blocked.
- So, the WDT counts down to zero and then it resets the whole MCU, so that the system recovers from its erroneous state.
- Count is decided in a WDT after considering how much time is to be allowed for the system to recover before it is forcibly reset.



2.6.4 REAL TIME CLOCK

- A real time clock inside the MCU is for providing calendar functions- the precise time in hours, minutes, seconds and also date required for various applications.
- All embedded systems need real time information.
- Even though a timer can be programmed to count time, it is best if there is a dedicated hardware which can be programmed.
- using its own registers to count 'real time' and present it easily for display and other purposes.
- Such a clock can also provide the timing for operating system, as high-end embedded systems use.
- It should also have a battery backup, if the clock is to be kept running all the time.

2.7 PULSE WIDTH MODULATOR

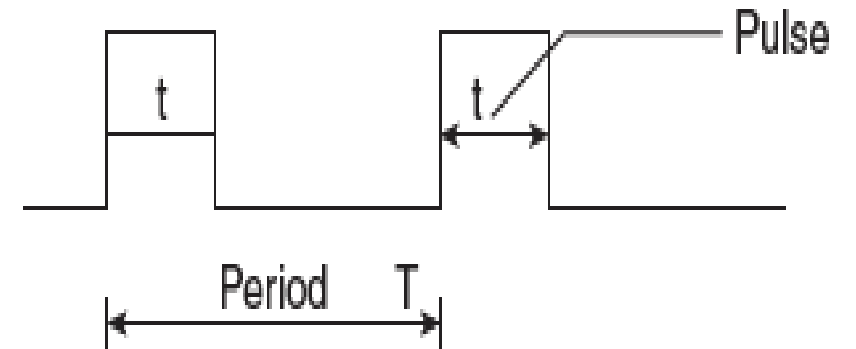
- PWM (Pulse Width Modulator) module is a timer. A Pulse Width Modulator (PWM) is a circuit or module that generates a pulse train whose pulse width can be varied, while the frequency usually remains constant.
- It is a separate module in MCUs.
- It is used to control power delivered to a load by adjusting the duty cycle of the signal i.e. Instead of reducing voltage directly (which wastes power), PWM switches ON and OFF rapidly to efficiently control power.
- Lets see what an PWM waveform is:

2.7 PULSE WIDTH MODULATOR

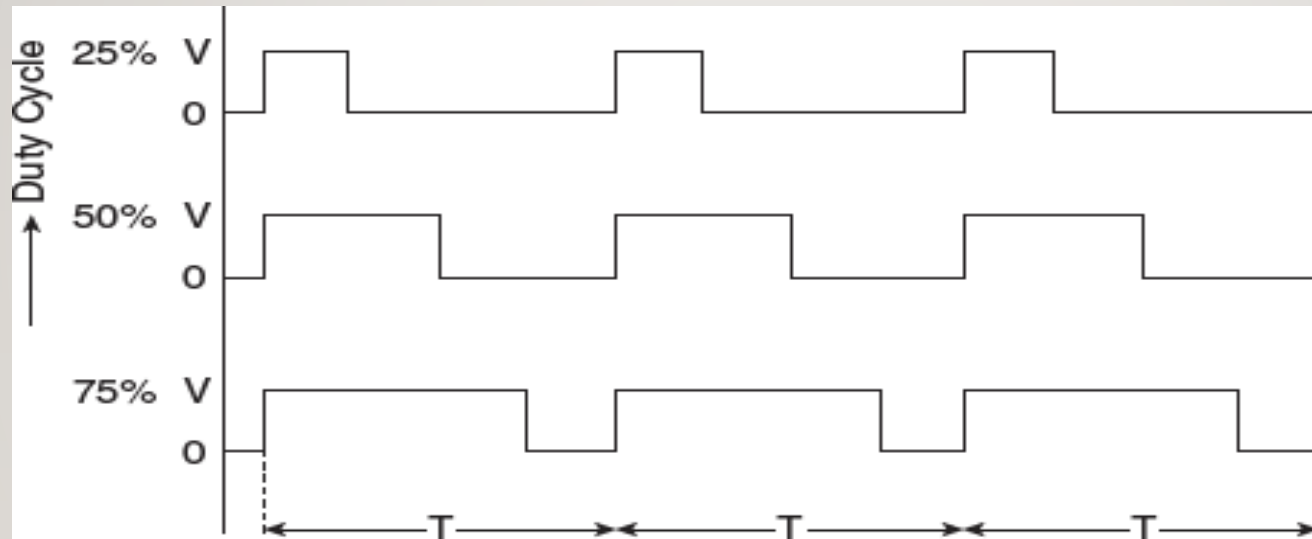
- A symmetric wave has half of its period in the high or '1' state and the other half in the low or '0' state.
- So its duty cycle is 50% where the duty cycle is defined as follows:
- if $T = 2t$
- Duty Cycle in % = (Time in the high state/Period) x 100

$$= (t/T) \times 100$$

$$= 50\%$$



-
- Asymmetric waveforms can have varying duty cycles.



- When we are able to change the duty cycle of a square wave, we say that it is “Pulse Width Modulated”.